AmigaED 4.0 Quick Manual

Quick Reference & Getting Started Guide

AmigaED is a cross-platform C/C++ and m68k Assembler IDE for classic Amiga development.

Table of Contents

3 Introduction 3

4 Two Ways to Build Your Code 4

5 Mode 1: Ad-hoc One File Compilation 5

6 Mode 2: Project Driven Compilation 6

7 The Preferences Editor 7

8 The Editor's Context Menu 8

9 Suggested Compiler & Linker Switches 9

10 Recommendations for Amiga C Programmers 10

AmigaED

Two Ways to Compile

A short, illustrated guide to Ad-hoc One File Compilation,
Project Driven Compilation, and the Preferences Editor

AmigaED is a cross-platform C/C++ and m68k Assembler IDE for classic Amiga development.

Two ways to build your code

AmigaED supports two independent ways to turn your source code into an AmigaOS-runnable program. Both are always available side by side - use whichever fits what you're doing right now.

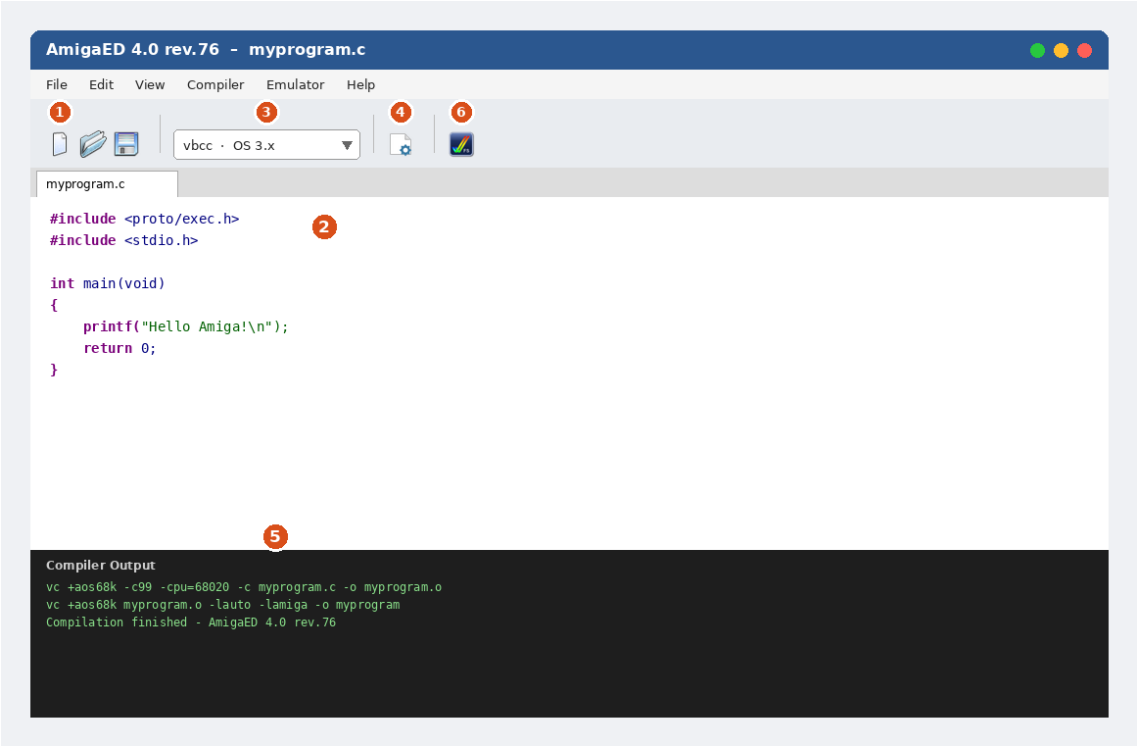
1. Ad-hoc One File Compilation – open a single source file and compile it directly, with no project setup at all. Best for a quick test, a one-off tool, or trying out an idea.

2. Project Driven Compilation – group multiple files (sources, headers, assembler, icons, ...) into a Project. AmigaED tracks them, generates Makefiles for you, and builds the whole thing with one click. Best for anything with more than one file, a GUI (ReAction/MUI), or that you'll come back to later.

	Ad-hoc	Project
Best for	Quick single-file tests	Multi-file / GUI programs
Setup needed	None - just open a file	Create or import a project
Multiple files linked together	No	Yes
Makefiles	Not generated	Auto-generated (gcc, vbcc, SAS/C)
Persists between sessions	No	Yes (.aep project file)

Mode 1: Ad-hoc One File Compilation

Use this when you just want to write and test a single source file, with no project, no linking multiple files, and nothing to set up first.



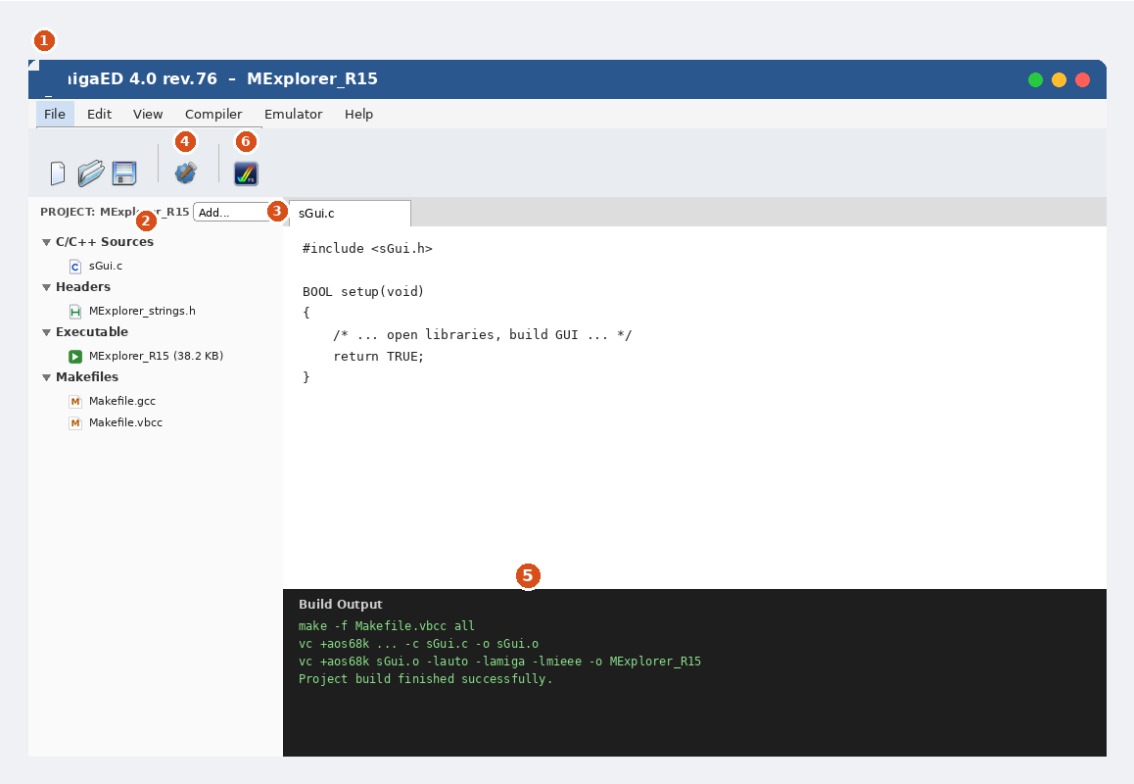
The editor in Ad-hoc mode: numbers match the steps below.

1. **File** → **New** (or **Open...**) to create or open a single `.c`/`.cpp` file. No project is involved.
2. Write or edit your code in the editor tab, with full syntax highlighting.
3. Pick your target **toolchain and OS** from the compiler dropdown in the toolbar (e.g. vbcc / OS 3.x, or m68k-amigaos-gcc / OS 1.3).
4. Click the **Compile** toolbar icon (or Compiler menu) to build just this file.
5. Check the **Compiler Output** panel at the bottom - it shows the exact compiler/linker commands used and whether the build succeeded.
6. Optionally, click **Start Emulator** to launch UAE and try your freshly built program right away.

Note: Ad-hoc compilation always builds and links just the one open file. If your program needs more than one source file, switch to Project Driven Compilation instead (see next chapter).

Mode 2: Project Driven Compilation

Use this for anything with more than one source file, a GUI (ReAction or MUI), or a program you'll want to rebuild and maintain over time.



The Project panel and toolbar: numbers match the steps below.

1 **File** → **New Project...** to start from a template (Empty C, AmigaShell, AmigaOS 1.3, AmigaOS 3.x, ReAction, or MUI), or **File** → **Import existing Project...** to bring in a folder of sources AmigaED doesn't know yet.

2 The **Project panel** on the left shows all your files, automatically sorted into categories - C/C++ Sources, Headers, Assembler Sources, AmigaGuide, Installer Scripts, Executables, Other Files, and the auto-generated Makefiles.

3 Add further files any time with **Add files to Project...**, or by dragging them straight onto the Project panel.

4 Click **Build Project**. AmigaED (re)generates the Makefiles for every configured toolchain and runs the build for you.

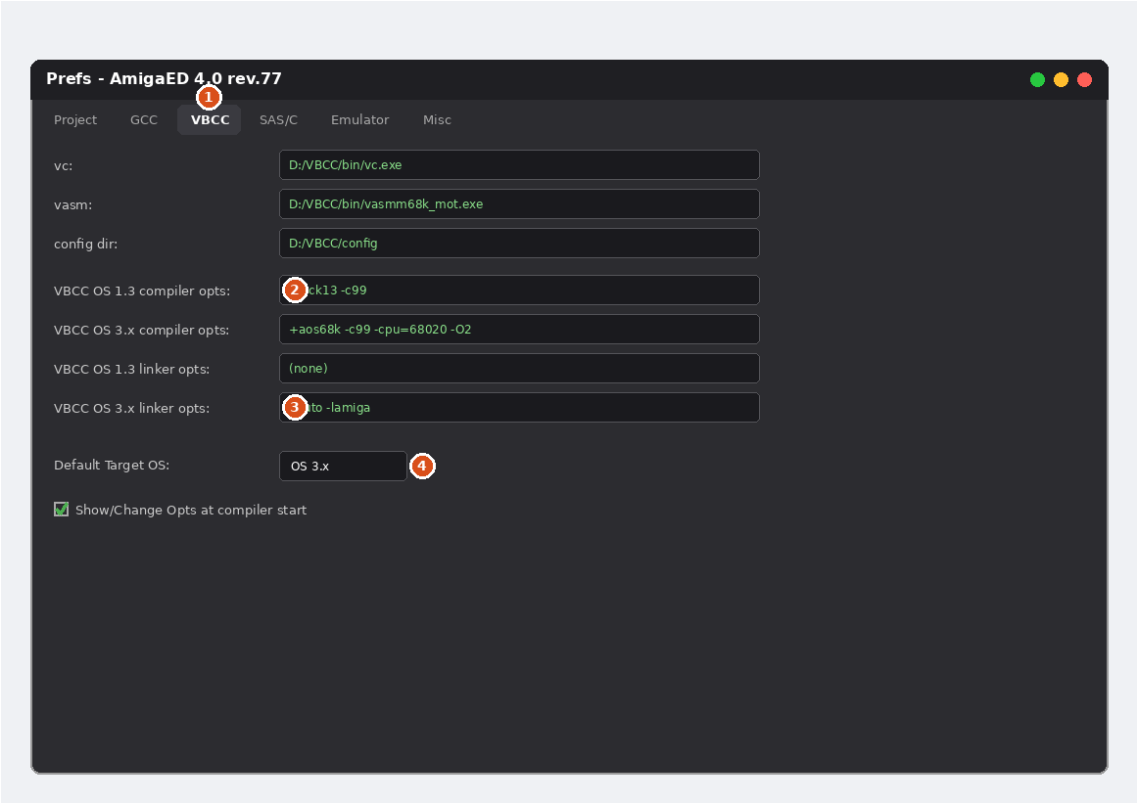
5 Watch the **Build Output** panel. On success, the compiled program appears in the Project panel's **Executable** category, together with its file size.

6 Click **Start Emulator** to launch UAE and test your program.

Note: A project's Makefiles are generated automatically whenever a file is added to or removed from the project - you never need to write or edit them by hand. Both vbcc and gcc/g++ Makefiles are generated side by side, plus a SAS/C Makefile for building later on a real Amiga.

The Preferences Editor

File → Prefs... opens AmigaED's settings, organized into tabs: **Project** (author/email/website, default projects folder), **GCC**, **VBCC**, and **SAS/C** (paths to each toolchain and their compiler/linker options - separate fields for AmigaOS 1.3 and AmigaOS 3.x targets), **Emulator** (path to your UAE executable), and **Misc** (application style/theme, default icon for new executables, and other editor behaviour).



The VBCC tab, as an example - GCC and SAS/C follow the same pattern.

1 Tabs across the top switch between Project, GCC, VBCC, SAS/C, Emulator, and Misc settings.

2 Compiler options are kept **separate for AmigaOS 1.3 and AmigaOS 3.x** - the two targets typically need different flags (see the tables below).

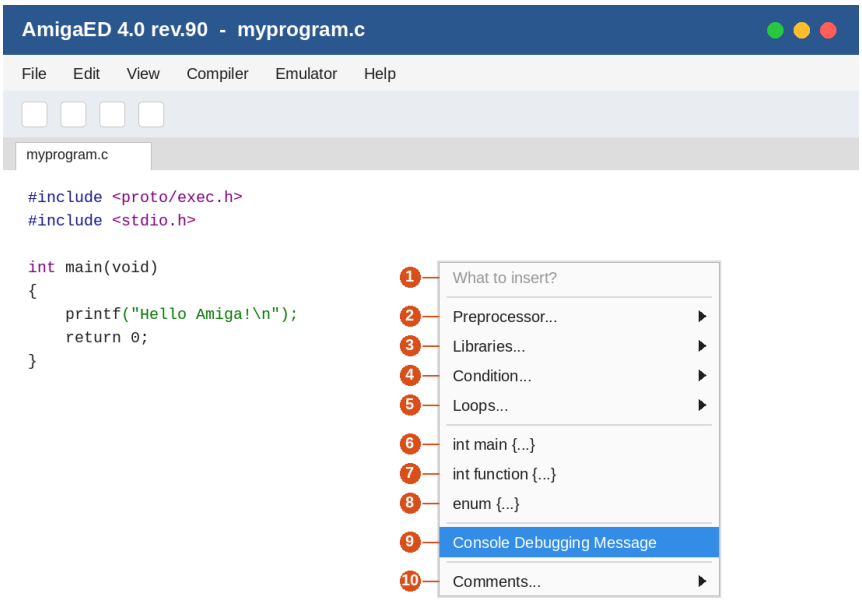
3 Linker options are likewise separate per target OS - this is where a math library gets added automatically when your project uses floating point (see note below).

4 **Default Target OS** sets which of the two option sets above is used when you don't explicitly pick one.

Automatic floating-point handling: if AmigaED detects `float` or `double` anywhere in your project's own C/C++ sources, it automatically adds the right math library to the generated Makefiles for you - `-lm` for gcc/g++, `-lmieee` for vbcc, and `MATH=IEEE` for SAS/C (only if no `MATH=` mode is already set). You don't need to add these yourself.

The Editor's Context Menu

Right-click anywhere inside the code editor to open a context menu of quick code-insertion shortcuts. It mirrors the main **Inserts** menu (menu bar) exactly - anything listed here is also available there, under the same name.



The editor's context menu, opened over a source file: numbers match the entries explained below.

1 **What to insert?** - a disabled heading line, not a clickable entry. It's only there to label the menu.

2 **Preprocessor...** - a submenu of preprocessor-related inserts: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if defined(...)`, **Amiga #include files** (a ready-made block of the most common Amiga NDK headers), **Identify Amiga compiler** (a chain of `#if defined(...)` checks generating a `compiler_string` constant naming whichever compiler is in use), and **Amiga C version string** (a `$VER:` tag).

3 **Libraries...** - `OpenLibrary()` and `CloseLibrary()`, each inserting a ready-to-edit `dummy.library` Open/Close template with error handling.

4 **Condition...** - `if(..) {...}` and `if(..) {...} else {...}` skeletons.

5 **Loops...** - `while(...) {...}`, `for(...) {...}`, `do...{...}while(...)`, and `switch(...)` (already including a `case dummy: break;`).

6 **int main {...}** - inserts a complete `main()` function skeleton; disabled once the file already contains one.

7 **int function {...}** - inserts a function prototype plus a matching function skeleton, ready to cut and paste into place.

8 **enum {...}** - inserts a complete C enumeration skeleton (`SomeEnum` with three example values).

9 **Console Debugging Message** - inserts an `if (myDebug) {...}` debugging block; the caret lands right inside it, ready to type. Needs `#define myDebug TRUE` earlier in the file - already included near the top of every "New Project" template.

10 **Comments...** - **Fileheader comment...**, C-style single/multi line comments, a C++ style single line comment, and a C-style line-dividing comment.

> **Note:** Most entries insert their skeleton right at the caret position and leave the caret ready to keep typing - either on a new line after the inserted block, or (for Console Debugging Message) right inside it. This context menu is built from the exact same actions as the main **Inserts** menu, so the two are always in sync.

Suggested compiler & linker switches

These are AmigaED's own built-in defaults - a solid, well-tested starting point for each toolchain and target. You're always free to add your own project-specific flags in the same Prefs fields alongside them.

m68k-amigaos-gcc (GCC/G++)

OS 1.3 compiler	`-Wall -O2 -mcrt=nix13`	Common warnings, optimize, link against libnix built for Kickstart 1.3.
OS 3.x compiler	`-Wall -O2 -noixemul`	Common warnings, optimize, use the AmigaOS-native C runtime instead of ixemul.library.
OS 1.3 linker	`(none)`	The OS 1.3 template is console-only and needs no extra libraries by default.
OS 3.x linker	`-lamiga`	Needed for Workbench-capable programs (ReAction/MUI) - added automatically by AmigaED's OS 3.x/ReAction/MUI templates.

vbcc (vc)

OS 1.3 compiler	`+kick13 -c99`	Named vbcc config for Kickstart 1.3, C99 language mode.
OS 3.x compiler	`+aos68k -c99`	Named vbcc config for AmigaOS 2.0+/3.x, C99 language mode.
OS 1.3 linker	`(none)`	Console-only OS 1.3 template needs no extra libraries by default.
OS 3.x linker	`-lauto -lamiga`	vbcc's own amiga.lib equivalent, needed for Workbench-capable (ReAction/MUI) programs.

Optional additions many users like to add for a release build once things compile cleanly: `-cpu=68020` (or higher, if you're deliberately targeting an accelerated Amiga - omit for maximum plain-68000 compatibility), `-O2`/`-O3` (optimize), `-size` (optimize for code size), `-final` (disable runtime checks meant for debugging). These aren't part of AmigaED's own defaults since they trade off compatibility or debuggability for speed/size - add them deliberately once you're happy with a working build.

SAS/C

SAS/C only runs on a real Amiga or emulator, so AmigaED never invokes it itself - it just generates a `Makefile.sc` alongside the others for later, manual use there. The SCOPTS field in Prefs starts empty; AmigaED still adds `MATH=IEEE` automatically if your project needs it (see above).

Recommendations for Amiga C Programmers

A short, curated list of literature and tools the Amiga C/ReAction/MUI development community particularly recommends - none of this is bundled with AmigaED, but all of it pairs well with it.

Literature

Amiga ROM Kernel Reference Manual: Exec / Libraries and Devices / Includes and Autodocs

The original Commodore-Amiga reference series - still the authoritative source for Exec, Intuition, and the rest of the classic system libraries. Out of print; free scanned copies are archived at archive.org.

Amiga ROM Kernel Reference Manual: AmigaDOS – Thomas Richter

A modern, RKRM-style reference specifically for AmigaDOS/dos.library, filling a gap the original series never fully covered. Free download from Aminet; a printed edition has also been available via lookbehindyou.de.

ROM Kernel Reference Manual: Changes & Additions – Camilla Boemann & Jason Stead

An ongoing (WIP) volume covering everything added to the Amiga OS from Release 2 through 3.2 that the original RKRM series predates - dedicated to original RKRM author Robert A. Peck. Free PDF from developer.amigaos3.net.

Erik Bartmanns Amiga Programmierbuch

C programming, cross-compiling with vbcc and ReAction, on an Amiga 4000 (or Amiga Forever). Book: 32€, e-book: 19.99€, via bombini-verlag.de.

Erik Bartmanns Amiga 1200 Buch

Machine language, C programming, and Shell - 717 pages across 28 chapters, focused on the Amiga 1200. Book: 64€, e-book: 29.99€, via bombini-verlag.de.

Tools

Rebuild – Darren Coles

A ReAction GUI builder (a from-scratch remake of the classic ClassMate), generating C or Amiga E code for windows, menus, and gadgets. Public domain. Source: github.com/dmcoles/ReBuild

Codecraft – Camilla Boemann

Codecraft is a powerful IDE for developing software natively on the Amiga. Codecraft makes it easy for you as a developer to write code, then build, execute and debug your resulting program. And everything is at your fingertips in one unified user interface. A native IDE for AmigaOS 3.2.3 and later. Source: gitlab.com/boemann/codecraft, http://boemann.dk/codecraft/

MUIBuilder

A user interface designer for the MUI and Zune toolkits, aiming to be portable across AmigaOS-like systems. GPLv3/LGPLv3. Source: sourceforge.net/projects/muibuilder

> **Note:** Prices, links, and availability (especially of print runs) change over time - if something above is no longer reachable, a web search for the title and author is usually the fastest way to find its current home.